Today

Welcome to your next lesson: Sync vs async.

I'm preparing your personalized lesson based on what you've learned, what's up next, and the best way to help you grow.

Step by step, you’re moving forward.

Ron, let’s start simple today.

In both Python and JavaScript, synchronous code runs one line at a time, in order — top to bottom. Nothing from line 3 can happen until line 2 is done.

Example in JavaScript:

Javascript
```
console.log("A");
console.log("B");
console.log("C");
```
This will always print in this order:

A

B

C

Because each console.log finishes before the next one starts.

Quick check 🎯

Does this idea of “one line at a time, in order” feel clear, or a bit fuzzy right now?

You can answer with:

“Clear”
“Fuzzy, explain again with another example”
Clear

Nice 👍

On the left of the visual I just showed, that’s synchronous: A → B → C, one after another, nothing overlapping.

Now look at the right side: we have Start, then a setTimeout(...) timer, then End. The timer counts in the background, and only when it’s done does the callback run and print Done. So the output order is:

Start → End → (after a pause) Done

Before we touch code, does that visual difference between the two sides make sense to you?

Synchronous vs Asynchronous – Simple Timeline

Yes, that makes sense

What do you think the correct order is? Take your time with the question below 👇

Given this JavaScript code:

```
javascript
console.log("Start");

setTimeout(function () {
  console.log("Done");
}, 1000);

console.log("End");
```

What order will the messages actually appear in the console?

A.
Start → Done → End

B.
Start → End → Done

C.
Done → Start → End

Exactly right 🎯

The console shows:

Start
End
after ~1 second → Done

Step by step:

console.log("Start"); runs immediately.
setTimeout(...) sets a timer and then JS moves on without waiting.
console.log("End"); runs next.
When the 1000 ms are up, the callback inside setTimeout finally runs and logs Done.

So you've just used asynchronous behavior correctly 💪

Next, I want you to see this yourself in the editor.

Open the code editor, paste this exact code, run it, and just watch the order in the console:

```
Javascript
console.log("Start");

setTimeout(function () {
  console.log("Done");
}, 1000);

console.log("End");
```

Run it (you don't need to submit yet), observe the timing, and tell me:

Did the output timing match what you expected?

yes vit matched
Start
End
Done

Perfect, that’s exactly what we expected 👌

Now let’s zoom out and name what just happened in simple terms:

The console.log("Start") and console.log("End") are synchronous work: run now, in order.
The setTimeout(..., 1000) creates asynchronous work: “run this function later, after at least 1000 ms, while I keep going with the rest of the code.”

A simple way to picture it:

Main road: where Start and End drive straight through.
Side road: the timer waits, then merges back in and prints Done when it’s ready.
I want to check that this idea is solid before we move on to your own example.

Can you confirm:

Does it feel clear that setTimeout does not pause the code, but instead schedules the callback to run later?

Yes, that’s clear

Great 👍

Now let’s focus on what setTimeout actually takes as its first argument.

In this code:

Javascript
```
setTimeout(function () {
  console.log("Done");
}, 1000);
```
The important part is that we are passing a function to setTimeout, not calling it right away.

Passing: setTimeout(myFunction, 1000); → “Run myFunction later.”
Calling: setTimeout(myFunction(), 1000); → This would run myFunction immediately and pass its result to setTimeout (which is not what we want).

So: setTimeout wants a function reference, not the result of calling that function.

Make sense so far, or should I go over the difference between myFunction and myFunction() one more time with a tiny example?

That makes sense

Awesome 👀

Let’s now make the event loop idea very concrete, with a simple mental picture.

Think of JavaScript as having two simple “areas”:

Main line → where your normal code runs, one line after another
Waiting area → where delayed tasks (like setTimeout) sit until their time is up

For your code:

Javascript
```
console.log("Start");

setTimeout(function () {
  console.log("Done");
}, 1000);

console.log("End");
```
Step by step:

console.log("Start") runs on the main line → prints Start
setTimeout(...) puts the callback into the waiting area for ~1000 ms, then JS keeps going
console.log("End") runs on the main line → prints End
After the delay, the callback comes back from the waiting area to the main line → prints Done

So the event loop is just:

"Is the main line free? Any finished waiting tasks? If yes, run their callback now."

Want to try a quick check on this idea with a small question?

Ask me a quick check

What do you think the order is here? Take your time with the question below 👇

Given this code:

javascript
```
console.log("First");

setTimeout(function () {
  console.log("Second");
}, 500);

console.log("Third");
```
What order will the messages appear in the console?

A.
First → Second → Third

B.
First → Third → Second

C.
Second → First → Third

First
Third
Second

Nice, you got it right again 🎯

For this code:

Javascript
```
console.log("First");

setTimeout(function () {
  console.log("Second");
}, 0);
```

console.log("Third");
The real order is:

First
Third
Second

Even with 0 ms, setTimeout still puts the callback into the waiting area to run after the current synchronous code (Third) finishes. That's the event loop in action.

Now I want you to build your own small async example to really lock this in

In the code editor, write JavaScript that does this, in order of lines:

Logs "A"
Uses setTimeout with a 500 ms delay to log "B"
Logs "C"

Run it (you can play a couple times if you want), observe the order, then Submit code and I'll review it.

Javascript
```
console.log("A");

setTimeout(function () {
  console.log("B");
}, 500);

console.log("C");
```
Nice work, Ron

Your code is exactly what I asked for, and the output you got:

A

C

(then after ~500ms) B

shows asynchronous behavior clearly:

console.log("A"); runs immediately.
setTimeout(..., 500); schedules "B" to run later, then JS keeps going.
console.log("C"); runs next.
After the delay, the callback runs and logs "B".

So you've now:

Written your own async example ☑
Seen the output order ☑
Quick check-in:

If you had to explain to someone why "C" appears before "B" here, do you feel ready to explain it in your own words, or would you like me to walk through it one more time using a little timeline diagram?

I can explain it

Love that

Let’s do one last thing to lock in all the pieces: sync vs async, setTimeout, and the “waiting area” / event loop idea.

Imagine this code:

Javascript

```
console.log("One");

setTimeout(function () {
  console.log("Two");
}, 1000);

console.log("Three");

setTimeout(function () {
  console.log("Four");
}, 500);

console.log("Five");
```

To check your understanding, I’ll give you a small multiple-choice question about the order next.

Ready to predict the output order for this one?

Ready—I'll predict it

What do you think the correct order is? Take your time with the question below 👇

Given this JavaScript code:

javascript

```
console.log("One");

setTimeout(function () {
  console.log("Two");
}, 1000);

console.log("Three");

setTimeout(function () {
  console.log("Four");
}, 500);

console.log("Five");
```

What is the actual order these messages will appear in the console?

A.
One → Two → Three → Four → Five

B.
One → Three → Five → Four → Two

C.
One → Three → Two → Five → Four

D.
One → Five → Three → Four → Two

Ron, that was spot on 🎯

For:

Javascript

```
console.log("One");

setTimeout(function () {
  console.log("Two");
}, 1000);

console.log("Three");

setTimeout(function () {
  console.log("Four");
}, 500);

console.log("Five");
```

The real order:

One → synchronous
Three → synchronous
Five → synchronous
Four → after ~500 ms (shorter timer finishes first)
Two → after ~1000 ms (longer timer finishes later)

So: One → Three → Five → Four → Two ☑

Why?

All normal console.log lines run first, top to bottom.
Both setTimeout calls go to the "waiting area".
The 500 ms timer finishes before the 1000 ms one, so its callback ("Four") runs first when the main line is free.

You've just shown you understand:

Sync vs async
How setTimeout works with callbacks
The simple "waiting area" / event loop idea
And you can predict output order for multiple timers 👀
To fully close this lesson, I want you to summarize in your own words:

In one or two sentences:

How would you explain the difference between synchronous and asynchronous JavaScript code to a friend who knows a bit of programming?

in synchronus code, each line of code runs one at a time, top to bottom.
With asynchronus code. the main syncchronus lines run one at time top to bottom while the asynchronus lines are ran in a side process, not executed until their timers have expired

That's a solid explanation, Ron 👏

Tiny refinement in your words (idea is already correct):

Synchronous: each line runs one at a time, top to bottom, and JS waits for each line to finish.
Asynchronous: JS starts some work (like a timer) and then keeps running the next lines; the async callback runs later, from the "side process" / waiting area, when its timer finishes and the main line is free.

You've shown you can:

Explain sync vs async in your own words ☑
Understand the simple event-loop / waiting-area idea ☑
Use setTimeout with a callback ☑
Write your own async example and reason about output order ☑

You've mastered everything for this lesson 🎉

Ready to wrap up this lesson and move on to the next one, or do you want to play with one more custom example first?

Move to the next lesson

Done ☑ You've officially completed Sync vs async.

Quick recap in plain language:

Synchronous JS: runs one line at a time, in order, and waits for each line to finish.
Asynchronous JS: can start a task (like a timer) and keep going; when the async task is ready, its callback runs later.
setTimeout(callback, delay):
takes a function reference as the first argument
schedules it to run after at least delay ms
You proved you understand this by:
Predicting output orders correctly
Writing your own setTimeout example
Explaining sync vs async in your own words

🔥 That's a core mental model for everything you'll do next with modern JavaScript.

When you're ready, hit Next Lesson in the Learn tab, and I'll keep building on this together with you.

```
console.log("First");

setTimeout(function () {
  console.log("Second");
}, 500);

console.log("Third");

console.log(A");

setTimeout(function () {
  console.log("B");
}, 500);

console.log("C");
```